Day 103 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 15, Day 6 reference

Watch: Lexical vs Semantic Search: BM25 vs Vectors (Kris Kassem) | Hybrid Search Finds What Vectors Miss (RAG) (Shane, LLM Implementation)

`day103_semantic_search.py`, segment by segment, in the order `main()` uses things. The encoder comes from the library; mean pooling, TF-IDF, BM25, rank fusion and the metrics are from scratch; Day 99's bootstrap and Day 102's questions are imported. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: f(w, d) = count of word w in paragraph d · |d| = paragraph length · k_1, b = BM25's constants · rank = position of the right paragraph (1 = top)

Segment 0 — Setup

Imports and settings

CODE

```
HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, HERE)
with contextlib.redirect_stdout(io.StringIO()):
    import day99_exercise as d99
    import day102_question_answering as d102
if hasattr(sys.stdout, "reconfigure"):
    sys.stdout.reconfigure(encoding="utf-8")
hf_logging.set_verbosity_error(); hf_logging.disable_progress_bar()
hub_logging.set_verbosity_error(); disable_progress_bars()
torch.set_num_threads(4)
ENCODER = "sentence-transformers/all-MiniLM-L6-v2"
KS = (1, 5, 10)
STOP = {"the", "a", "an", "of", "in", "to", "is", "was", "and", "for", "on", "by", "with", "as",
    "at", "that", "from", "are",
        "were", "it", "its", "be", "this", "or", "which", "what", "who", "when", "where", "how",
            "why", "did", "does", "do"}
```

- `ENCODER = "sentence-transformers/all-MiniLM-L6-v2"` — a model trained so that similar sentences get similar vectors.
- `STOP = {...}` — a set of common words ignored by the sparse methods.

Segment 1 — Dense search

mean_pool and embed: one unit vector per text

MATH

$$v = \Sigma_t m_t h_t / \Sigma_t m_t, \quad cos(q, d) = q \cdot d / (|q| \, |d|)$$

CODE

```
def mean_pool(hidden, mask):
    m = mask.unsqueeze(-1).float()
    return (hidden * m).sum(1) / m.sum(1).clamp(min=1e-9)
def embed(tok, enc, texts, bs=32, max_len=256):
    out = []
    with torch.no_grad():
        for i in range(0, len(texts), bs):
            b = tok(texts[i:i + bs], padding=True, truncation=True, max_length=max_len,
                return_tensors="pt")
            v = mean_pool(enc(**b).last_hidden_state, b["attention_mask"])
            out.append(torch.nn.functional.normalize(v, dim=-1).numpy())
    return np.concatenate(out)
def ranks_of(scores, gold):
    g = scores[np.arange(len(gold)), gold]
    return 1 + (scores > g[:, None]).sum(1)
```

- `mask.unsqueeze(-1).float()` — adds a last axis of size 1, so the mask multiplies every dimension of each token vector.
- `m.sum(1).clamp(min=1e-9)` — the number of real tokens, never zero.
- `torch.nn.functional.normalize(v, dim=-1)` — divides each vector by its length: then a dot product is a cosine.
- `scores[np.arange(len(gold)), gold]` — fancy indexing with two arrays: each query's score for its own right paragraph.

Segment 2 — Sparse search and fusion

BM25: rarity, saturation, length

MATH

$$score(q, d) = \Sigma_{w \text{ in } q} \, idf(w) \cdot f(w,d)(k_1 + 1) / (f(w,d) + k_1(1 - b + b|d|/avgdl))$$

CODE

```
class BM25:
    def __init__(self, docs, k1=1.5, b=0.75):
        self.tf = [Counter(words(d)) for d in docs]
        self.len = np.array([sum(t.values()) for t in self.tf])
        self.avg, self.k1, self.b, self.N = self.len.mean(), k1, b, len(docs)
        df = Counter(w for t in self.tf for w in t)
        self.idf = {w: np.log(1 + (self.N - n + 0.5) / (n + 0.5)) for w, n in df.items()}
    def scores(self, query):
        s = np.zeros(self.N)
        norm = self.k1 * (1 - self.b + self.b * self.len / self.avg)
        for w in set(words(query)):
            if w in self.idf:
                f = np.array([t.get(w, 0) for t in self.tf])
                s += self.idf[w] * f * (self.k1 + 1) / (f + norm)
        return s
```

- `np.log(1 + (self.N - n + 0.5) / (n + 0.5))` — idf: a word in few paragraphs counts more; the +1 inside keeps it positive.

- `self.k1 * (1 - self.b + self.b * self.len / self.avg)` — longer-than-average paragraphs need more matches for the same score.
- `f * (self.k1 + 1) / (f + norm)` — saturation: the score grows with the count but levels off.

tfidf_matrix and rrf

MATH

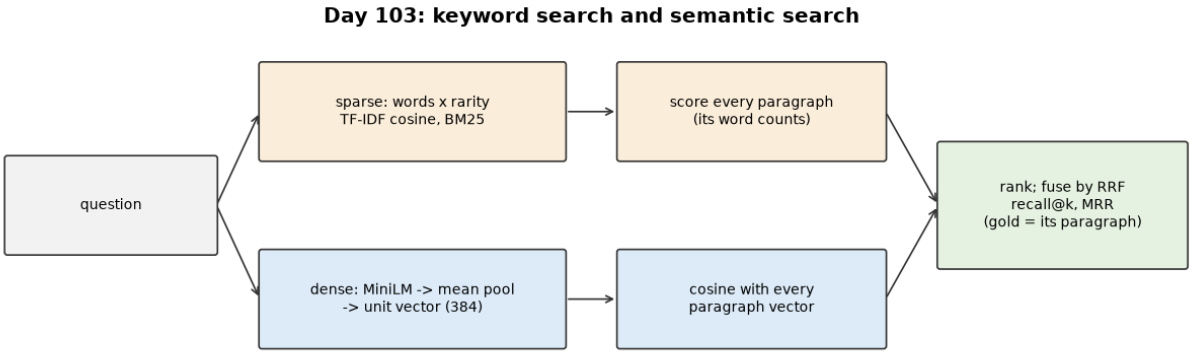
$$RRF(d) = \Sigma_{systems}\ 1 / (60 + rank_{system}(d))$$

CODE

```
def tfidf_matrix(docs, queries):
    vocab = {w: i for i, w in enumerate(sorted({w for d in docs for w in words(d)}))}
    def vec(texts):
        M = np.zeros((len(texts), len(vocab)), dtype=np.float32)
        for r, t in enumerate(texts):
            for w, c in Counter(words(t)).items():
                if w in vocab:
                    M[r, vocab[w]] = c
        return M
    D, Q = vec(docs), vec(queries)
    idf = np.log(len(docs) / (1 + (D > 0).sum(0))) + 1
    D, Q = D * idf, Q * idf
    return D / (np.linalg.norm(D, axis=1, keepdims=True) + 1e-12), Q / (np.linalg.norm(Q, axis=1,
        keepdims=True) + 1e-12)
def rrf(*score_mats, k=60):
    total = 0.0
    for S in score_mats:
        r = np.argsort(np.argsort(-S, axis=1), axis=1) + 1
        total = total + 1.0 / (k + r)
    return total
```

- `def vec(texts): ...` — a function defined inside another: it can use vocab from the enclosing function.
- `np.argsort(np.argsort(-S, axis=1), axis=1) + 1` — sorting twice turns scores into ranks (1 = best) row by row.
- `total + 1.0 / (k + r)` — reciprocal rank fusion: high ranks in either list count; k = 60 damps the top.

SYSTEM DIAGRAM



Keyword search and semantic search, side by side, fused by rank.

Segment 3 — Measuring

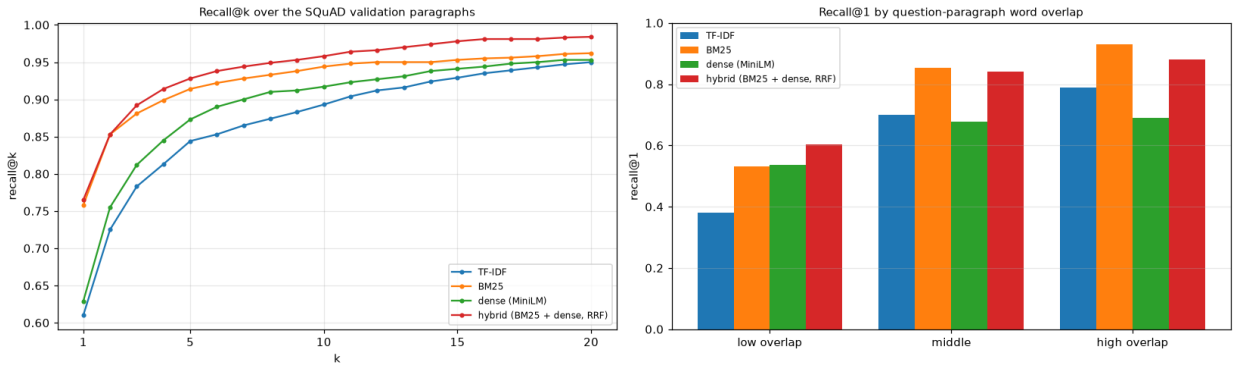
demo_metrics: recall@k, MRR, same-article hits

CODE

```
tarr = np.array(titles)
for name, r in R.items():
    hit1 = (r == 1).astype(float); lo, hi = d99.bootstrap_ci(hit1)
    top = S[name].argmax(1)
    same = np.mean(tarr[top] == tarr[gold])
    print(f"{name:<28}" + "".join(f"{np.mean(r <= k):>8.3f}" for k in KS) + f"{np.mean(1 /
        r):>8.3f}      [{lo:.3f}, {hi:.3f}]{same:>17.3f}")
print("\npaired differences in recall@1 (Day 99): mean, 95% interval, share of resamples ahead")
for a, b in (("dense (MiniLM)", "BM25"), ("hybrid (BM25 + dense, RRF)", "BM25"), ("hybrid (BM25 +
    dense, RRF)", "dense (MiniLM)"),
             ("BM25", "TF-IDF")):
    m, lo, hi, share = d99.paired_bootstrap((R[a] == 1).astype(float), (R[b] == 1).astype(float))
    print(f"  {a:<27} minus {b:<16} {m:+.3f} [{lo:+.3f}, {hi:+.3f}] ahead in {share:.3f}")
```

- `np.mean(r <= k)` — recall@k: the share of questions whose right paragraph ranks k or better.
- `np.mean(1 / r)` — mean reciprocal rank: 1 for first place, 1/2 for second, and so on.
- `tarr[top] == tarr[gold]` — the first result is from the right article even when it is the wrong paragraph.

REAL GRAPH



Real graph from the script: recall@k for each search, and recall@1 by word overlap.

Quick Syntax Glossary — new or reinforced constructs

- `tensor.unsqueeze(-1)` — add an axis of size 1 at the end.
- `tensor.clamp(min=x)` — replace values below x with x.
- `F.normalize(v, dim=-1)` — scale vectors to length 1.
- `np.argsort(np.argsort(x))` — ranks from scores.
- `dict.fromkeys(list)` — distinct items in order of first appearance.
- `def outer(): def inner(): ...` — a nested function that sees the outer function's variables.

End of Day 103 syntax reference. For the experiments and what they showed, see the Day 103 complete guide.